

Laboratorio di programmazione

[Appunti di Piai Luca]

Academic Year 2022-2023

Indice

1	Dichiarazioni, scope e librerie	4
1.1	Istruzioni, dichiarazioni e definizioni	4
1.2	Scope e namespace	4
1.3	Librerie e linking	4
2	Funzioni e argomenti, mappa della memoria	4
2.1	Funzioni e reference, efficienza	4
2.2	Chiamate a funzione e memory layout	5
3	Classi e overloading	5
3.1	Le classi	5
3.2	Enumerazioni	6
3.3	Overloading degli operatori	6
4	Puntatori, array e gestione della memoria	6
4.1	I puntatori	6
4.2	Puntatori, reference, cast	7
4.3	Array	7
4.4	Allocare e deallocare la memoria usando il free store	8
5	Classi: inizializzazione, copia, spostamento, accesso	8
6	Gestione dello sviluppo di progetti	11
6.1	Organizzazione del software in progetti complessi	11
6.2	Introduzione a CMake	11
7	Template	12
8	STL: Standard Template Library	13
9	Ereditarietà e funzioni virtuali	15
9.1	Ereditarietà	15
9.2	Funzioni virtuali	16
9.3	Virtual pointer e virtual table	16
9.4	Navigare nella gerarchia di classi	17
9.5	Polimorfismo	17
10	Introduzione agli algoritmi STL	18
10.1	Un primo esempio: find()	18
10.2	Algoritmi con predicati: find_if	18
10.3	sort(), copy() e lambda expression	19
10.4	Associative container	21
11	Smart pointer	22
11.1	Risorse ed eccezioni	22
11.2	Smart pointer	23

1 Dichiarazioni, scope e librerie

1.1 Istruzioni, dichiarazioni e definizioni

Le istruzioni sono parte di codice che specifica un'azione. Sono escluse le direttive (`#include`). Tutte le istruzioni terminano con un punto e virgola. Alcuni esempi sono: expression statements, dichiarazioni, selezione, iterazione.

Le dichiarazioni specificano un nome, un tipo e (opzionalmente) una inizializzazione. Le definizioni sono dichiarazioni che specificano completamente le entità dichiarate. Per esempio: `double sqrt(double);` è una dichiarazione; `double sqrt(double){...}` è una definizione.

1.2 Scope e namespace

Lo scope è la regione di testo di un programma. Le variabili sono valide solo all'interno dello scope in cui vengono dichiarate. Esistono diversi scope all'interno di un programma: globale, di classe, namespace, locale, di statement (per esempio nei for-statement).

I namespace sono strumenti di scoping. Permettono di organizzare dati, funzioni e classi in uno stesso scope che definiamo noi.

Per esempio: `namespace Graph { class Color { ... } }`, ora la classe si chiama `Graph::Color`. Se nel programma in cui voglio utilizzare la classe aggiungo `using graph::Color`, non serve più il prefisso con il nome del namespace. In alternativa con `using namespace Graph` possiamo utilizzare direttamente i nomi di tutte le identità presenti.

1.3 Librerie e linking

Le librerie sono uno strumento che ci permette di importare funzioni scritte da altri.

Progetti grandi richiedono molte dichiarazioni e molte definizioni, per questo motivo utilizziamo i file header come spazio per dichiarare entità, le definizioni vengono inserite in un altro file sorgente (estensione `.cpp`).

Per utilizzare le entità dichiarate bisogna importare il file header nel progetto. Utilizziamo: `#include <iostream>` per importare gli header di sistema; `#include "My_header.h"` per importare header dell'utente. Nota che la compilazione del file sorgente contenente le definizioni è necessaria solo per le librerie scritte da noi, le librerie standard sono già precompilate per questioni di efficienza.

Esistono due tipi di librerie: statiche e dinamiche. Se una libreria è statica: ogni file oggetto viene linkato con la libreria precompilata che necessita. Il vantaggio è che gli eseguibili non possono essere spezzati, inoltre, essendo self-contained, sono più adatte alla distribuzione di software monolitico (autonomo). Se la libreria è dinamica allora più file oggetto fanno riferimento alla stessa libreria precompilata, l'eseguibile prodotto non contiene la libreria. Queste librerie possono essere ricomilate senza toccare gli eseguibili (su linux prendono il nome di Shared Objects, su windows Dynamic Linking Library) inoltre occupano meno spazio su disco (una copia funziona per tutti gli eseguibili).

2 Funzioni e argomenti, mappa della memoria

2.1 Funzioni e reference, efficienza

Quando passiamo parametri a una funzione possiamo passarli per copia oppure per riferimento. Il passaggio per copia è poco efficiente per oggetti molto grandi, in questi casi si utilizzano le reference.

Attraverso le reference accediamo all'indirizzo di memoria in cui si trova il dato, quindi passando un parametro per reference ad una funzione, stiamo di fatto passando un indirizzo,

Questo meccanismo ci permette di evitare la copia del dato.

Esempio: `void func(vector<int>& v);`.

Questo tipo di scrittura non protegge dalla sovrascrittura, cioè la funzione potrebbe modificare i valori del vettore passato. Per ovviare a questo problema usiamo la scrittura `void func(const vector<int>& v);`, in questo modo la funzione può accedere in sola lettura e non si rischia di compromettere i dati passati (funziona anche con i tipi primitivi).

Le reference vengono usate anche per riferirci ad altre variabili.

Per esempio: `int a = 22; int& var = a;`, in questo modo se modifico `var` modifico di conseguenza `a` (obbligatorio inizializzare le reference!).

2.2 Chiamate a funzione e memory layout

Chiamare una funzione causa la creazione di una struttura dati RA (record di attivazione) che contiene: una copia dei parametri; variabili locali. Gli RA vengono impilati in una zona della memoria chiamata stack, la dimensione degli RA dipende dal numero e tipo di variabili locali che contiene la funzione.

La memoria è composta da diverse parti:

- Text segment: copiato in memoria dal file oggetto; contiene le istruzioni eseguibili.
- Initialized data segment: contiene variabili globali e variabili statiche.
- Uninitialized data segment: nota come BSS, viene inizializzata a 0 dal sistema operativo, contiene variabili globali e statiche non inizializzate esplicitamente.
- Stack: contiene i RA delle funzioni (variabili locali e parametri).
- Heap: free storage.

3 Classi e overloading

3.1 Le classi

Le classi ci permettono di creare nuovi tipi (UDT). Le entità che compongono una classe sono: data member e function member. Se non specificato vengono dichiarate `private` di default. Per accedere ai membri di una classe utilizziamo la notazione `obj.func();`.

Le strutture sono simili alle classi ma i membri sono pubblici di default. Spesso vengono utilizzate per definire gruppi di variabili accessibili a tutti. Derivano dal linguaggio C.

Vediamo una carrellata di caratteristiche e termini generali riferite alle classi:

- Helper function: funzioni che eseguono operazioni comuni, per esempio la verifica della validità dei dati oppure l'inizializzazione. Sono molto utili, sono preferibili alle function member.
- Funzioni membro: garantiscono che i dati membro siano sempre coerenti, in altre parole che l'oggetto sia sempre in uno stato valido. Possono essere definite dentro la classe o all'esterno: se le definisco all'esterno evito di mescolare i dettagli implementativi all'interfaccia, solitamente vengono definite all'interno solo le funzioni molto brevi (get, set).
- Invariante: sono le regole per definire i valori validi dello stato degli oggetti di una classe (variano in base alla classe). Deve rimanere sempre vera.

- Costruttori: inizializzano gli oggetti con valori validi. Spesso si utilizzano le liste di inizializzazione (es: `Date::Date(int yy, int mm, int dd): y{yy}, m{mm}, d{dd} {}`) in quanto sono molto più efficienti (un passaggio in meno rispetto al costruttore in cui inizializzo manualmente ciascuna variabile).
- Eccezioni ed errori: per verificare che i parametri passati ai costruttori siano validi si può definire una funzione privata `bool is_valid()`. All'interno dell'interfaccia pubblica dichiariamo una classe `class Invalid{}`; questa ha il solo scopo di creare un tipo unico che segnala uno specifico errore. Se `bool is_valid()` torna `false`, il costruttore lancia la classe `Invalid` con la sintassi `throw Invalid();`. Le eccezioni vengono gestite con i `try` e `catch`.
- Funzioni membro costanti: sono funzioni che esplicitamente non modificano lo stato di un oggetto. In questo modo possiamo invocare funzioni che consentono l'accesso ai membri con oggetti passati come `const` reference. Un esempio di funzione membro costante: `int get_day()const { return day; }`

3.2 Enumerazioni

Le enumerazioni sono un tipo definito dall'utente, specificano un set di valori. Il corpo è composto dalla lista di valori. Per esempio: `enum class Month { jan = 1; feb, mar, ..., dec };` in questo caso abbiamo specificato che la lista parte dal valore 1. Possiamo creare oggetti, per esempio: `Month m = Month::feb` oppure `Month m = Month(2);`.

Le enumerazioni possono avere lo scope di classe (come nell'esempio), oppure possono essere globali (rimuovendo il prefisso `class`), in questo possiamo scrivere `Month m = feb;`.

3.3 Overloading degli operatori

In C++ è possibile implementare quasi tutti gli operatori per operandi di tipo definito dall'utente (`class` o `enum`).

L'overloading viene implementato creando una funzione con un nome specifico: `operator++`, `operator+`, `operator[]`, `operator<<`, `operator>`, ecc. Tra i parametri passati deve esserci almeno un argomento UDT.

Possono essere definiti attraverso helper function oppure funzioni membro. La differenza è che se implementati con helper function richiedono due argomenti, come funzioni membro un argomento solo in quanto il secondo è implicito.

Di solito è preferibile l'implementazione come helper function, spesso vengono inserite in un namespace insieme alla classe.

Esistono tre categorie di operatori: unari (es: `++`, `--`), binari (es: `+`, `<`, `||`), e ternari (es: `?:`).

4 Puntatori, array e gestione della memoria

4.1 I puntatori

I puntatori sono un particolare tipo di variabile che contiene un indirizzo di memoria. La definizione di un puntatore contiene: il tipo puntato; il carattere `*` che significa che la variabile definita è un puntatore. Vediamo degli esempi:

```

1 double* p; // dichiaro un puntatore
2 int* ptr = &var; // acquisisco l'indirizzo di una variabile
3 *ptr = 22; // operatore dereference accedo al contenuto della memoria puntata
   e ci scrivo

```

```
4 cout << *ptr << ptr; // stampo il valore contenuto nella cella puntata e
   l'indirizzo contenuto nel puntatore
5 double* p0 = nullptr; // inizializzo un puntatore non valido (equivalente a 0
   o null)
6 sizeof(int) // stampa la dimensione di un int in memoria
7 sizeof(int*) // stampa la dimensione di un puntatore a un int in memoria
```

Quando si utilizzano puntatori bisogna stare particolarmente attenti. Essi sono una causa frequente di errori runtime difficili da individuare se non con l'utilizzo di un *debugger* (per esempio gdb su linux). Tra i problemi più comuni troviamo:

- accesso ad un `nullptr`;
- accesso a un puntatore non inizializzato;
- accesso fuori dai limiti di un array;
- accesso ad un oggetto deallocato;
- accesso ad un oggetto out of scope.

4.2 Puntatori, reference, cast

Con `void*` posso dichiarare un puntatore che salta il controllo sul tipo, non specifico nulla sull'oggetto puntato. Rappresenta il concetto di puro di indirizzo di memoria senza indicazioni su come usarla. Non si può dereferenziare, ma usando il cast esplicito `static_cast<T> (var)` possiamo convertire anche questo tipo di puntatori, questo tipo di cast "forzato" è utile per interfacciarsi con HW o altro codice non modificabile.

Le reference sono come puntatori con la differenza che sono immutabili (non possiamo spostare una reference) e vengono dereferenziati automaticamente. Inoltre, a differenza delle reference, i puntatori hanno la possibilità di essere inizializzati a `nullptr`, questo risulta utile per il passaggio di parametri per puntatori.

In generale se il no-object è un valore plausibile è meglio passare un puntatore come parametro, in questo modo ci basta verificare che non sia `nullptr`. Per gli altri casi possiamo usare reference o passaggio per copia.

4.3 Array

Gli array sono una struttura dati più semplice rispetto agli `std::vector`, ma sono molto utili in alcune circostanze che vedremo più avanti. La definizione ci dice che un array è una sequenza omogenea di oggetti allocati in spazi di memoria contigui.

Inoltre: hanno una dimensione fissa nota a tempo di compilazione (literal o `const`); non avviene nessun controllo su lettura o scrittura fuori range; vengono trasformati come puntatori se passati come argomenti a una funzione (quindi non abbiamo informazioni sulla dimensione).

Gli array sono puntatori che puntano al loro primo elemento, tra puntatori e array c'è una particolare relazione. Analizziamo alcune righe:

```
1 int a[10] = {}; // array di dieci elementi inizializzati a 0
2 func(a); // equivalente a func(&a[0])
3 int* p = &a[4] // puntatore ad un elemento di un array
4 *p = 7; // assegno scrivo un valore nella cella
5 p[2] = 6; // scrivo il valore 6 nella cella 4+2
6 p[-3] = 7; // scrivo il valore 7 nella cella 4-3
7 int* pd = a; // puntatore che punta alla prima cella di a
```

```
8 | pd+=3; //ora punta alla cella 0+3
```

Le stringhe in stile C sono degli array di `char`. C++ offre una versione più moderna e di alto livello: `std::string`.

```
1 | char name[] = "Luca"; // se inizializzo con le grafe devo aggiungere il
   | terminatore '\0'
2 | char* ptr = "Mario"; // puntatore allo spazio di memoria che contiene
   | "Mario" (equivalente di una stringa)
```

4.4 Allocare e deallocare la memoria usando il free store

Allocazione:

Il free store (o heap) è uno spazio di memoria che possiamo utilizzare per allocare e deallocare dinamicamente oggetti. In questo modo possiamo creare delle strutture dati dinamiche.

Molte entità della libreria standard sfruttano l'allocazione dinamica della memoria, per esempio: `std::vector`, `std::string`, ecc.

Per accedere al free store usiamo l'operatore `new` oppure `new[]`, questo ritorna un puntatore allo spazio di memoria allocato. Per esempio: `double* ptr = new double[4];` alloca `4*sizeof(double)` nel free store e ritorna l'indirizzo del primo elemento.

Questa operazione richiede tempo in quanto viene fatta runtime: il programma comunica con il sistema operativo per allocare lo spazio richiesto.

Riassumendo: `new` alloca la memoria poi invoca il costruttore.

Deallocazione:

Per deallocare la memoria nel free store usiamo l'operatore `delete` oppure `delete[]`.

Se decidiamo di spostare un puntatore che puntava a un oggetto nella memoria dinamica, dobbiamo ricordarci di: deallocare la memoria, altrimenti creiamo garbage (i memory leak sono errori gravi del programmatore); settare il puntatore a `nullptr`, punta a una zona di memoria che non gli compete più.

Anche `std::vector` della libreria standard deve deallocare la memoria quando un oggetto raggiunge il fine vita, lo fa usando un distruttore: una funzione che viene chiamata implicitamente dal compilatore. Viene chiamata quando un oggetto deve essere distrutto, cioè quando esce dallo scope o viene deallocato.

Nota che il distruttore (se non viene implementato) è generato di default dal compilatore.

Riassumendo: `delete` invoca il distruttore poi libera la memoria.

5 Classi: inizializzazione, copia, spostamento, accesso

Le operazioni essenziali da considerare per una classe sono:

- costruttore con uno o più argomenti: il costruttore con un argomento definisce una conversione da quell'argomento alla classe. In alcuni casi, questo comportamento potrebbe essere indesiderato (per esempio `vector v = 10; //crea un vector di 10 elementi`), in questi casi si usa la keyword `explicit` anteposta al costruttore in questione (`explicit vector(int);`).
- costruttore di default;
- costruttore di copia (copy constructor);
- assegnamento di copia (copy assignment);

- costruttore di spostamento (move constructor);
- assegnamento di spostamento (move assignment);
- distruttore: necessario se acquisiamo risorse (che devono essere liberate), probabilmente è necessario se ci sono membri puntatori.

Nota che: se nella classe non viene specificato nessun costruttore, ne viene creato uno di default; se non viene implementato un copy constructor e un copy assignment, ne vengono forniti due di default; move constructor e assignment non vengono mai forniti di default; il distruttore viene fornito di default se non viene implementato.

Vediamo i vari punti nello specifico prendendo come esempio un'implementazione della classe `std::vector`.

Inizializzazione di oggetti:

Vogliamo poter inizializzare gli oggetti con una scrittura del tipo `vector v1 = { 1.2, 7.89, 12.34 };`. Una scrittura di questo tipo fornisce un oggetto della standard library di tipo `initializer_list<T>`, dobbiamo aggiungere nella nostra implementazione un nuovo costruttore:

```
1  class vector {
2      // ...
3      vector(initializer_list<double> lst)
4          : sz{lst.size()}, elem{new double[sz]}{
5          copy(lst.begin(), lst.end(), elem); // copy(from, to, ptr)
6      }
7      // ...
8  };
```

Nota che `initializer_list<double> lst` è passato per copia, come richiesto dal linguaggio.

Copia di oggetti:

Di default esiste un costruttore di copia, questo effettua una shallow copy: copia di puntatori o reference senza copiare i dati a cui puntano (comportamento indesiderato).

Può essere utile definire una deep copy: copia dei dati; è necessario definire un costruttore di copia e un assegnamento di copia. (Comportamento predefinito di `std::vector`, `std::string`, ecc).

```
1  class vector {
2      int sz;
3      double* elem;
4  public:
5      vector(const vector& arg) // definiamo un costruttore di copia
6          : sz{arg.sz}, elem{new double[arg.sz]} {
7          copy(arg.elem, arg.elem+sz, elem);
8      }
9
10     vector& operator=(const vector& a); // assegnamento di copia in cui
11         deallochiamo i vecchi dati puntati da elem e allochiamo i dati di a
12         //...
13 };
```

L'assegnamento è leggermente diverso dal costruttore, deve gestire i dati vecchi.

Nota che se `vector` non implementasse il costruttore e l'assegnamento di copia (e quindi ci sarebbe la shallow copy di default) e scrivessimo `vector a = v;`, non solo i due puntatori

`elem` andrebbero a puntare alla stessa locazione di memoria ma (usciti dallo scope e con un distruttore appropriato) andrebbero entrambi a deallocare, causando quindi un errore runtime.

Spostamento di oggetti:

Cerchiamo un'operazione di move che complementi le operazioni di copia. Analizziamo un esempio:

```

1  vector fill(istream& is) {
2      vector res;
3      for (double x; is >> x; ) res.push_back(x); // inserire dati => true
4      return res;
5  }
6
7  void use() {
8      vector vec = fill(cin);
9      // uso di vec
10 }
```

Il problema è che andiamo ad usare una copia del vector che viene ritornato dalla funzione. Vogliamo un modo per scrivere "direttamente" nel vettore `vec`, in questo modo miglioriamo le prestazioni del programma.

Introduciamo il move constructor e il move assignment:

```

1  class vector {
2      int sz;
3      double* elem;
4  public:
5      vector(vector&& a); // move constructor
6      vector& operator=(vector&& a); // move assignment
7      // ...
8  };
```

La notazione `&&` è chiamata rvalue reference. Il costruttore e l'assegnamento di copia permettono di spostare oggetti da uno scope ad un'altro e permettono di risparmiare memoria.

Nota che in certe circostanze il move constructor non viene chiamato, viene sostituito da una tecnica di ottimizzazione del compilatore chiamata copy elision (crea l'oggetto direttamente nella funzione chiamante).

Accesso ai membri di una classe:

Vogliamo poter accedere (leggere e scrivere) agli elementi dei vettori tramite le parentesi quadre.

```

1  class vector {
2      int sz;
3      double* elem;
4  public:
5      double& operator[] (int n) { return elem[n]; } // (non utilizzabile per
6      double operator[] (int n) const; // torna una copia o una const reference
7      // ...
8  };
```

Il primo overload ritorna una reference, questa ci fornisce sia l'accesso in lettura che l'accesso in scrittura. Nota che `elem` è un data member, è privato, è sbagliato tornare una

reference di un dato privato? Non c'è una risposta, in ogni caso sarebbe molto difficile implementare la scrittura `v[0]` senza tornare la reference dell'elemento.

Il secondo overload ci permette di utilizzare l'operatore `[]` con gli oggetti `const`. In questo caso non abbiamo il problema della scrittura, non ci serve un `lvalue` in nessun caso.

6 Gestione dello sviluppo di progetti

Elenchiamo alcune pratiche utili per gestire progetti in cui collaborano più persone.

6.1 Organizzazione del software in progetti complessi

I progetti software complessi devono essere organizzati in più file. Dividiamo i file in due fondamentali categorie:

- file header: contengono definizioni di classe e dichiarazioni di funzione. Vengono inclusi con `#include` e non devono essere compilati direttamente;
- file sorgente: contengono definizioni di funzione e il `main` (unico). Vengono compilati dal compilatore, non devono essere inclusi.

Un file sorgente può includere più header, gli header possono includere a loro volta altri header. Questo meccanismo può generare delle inclusioni cicliche.

Per questa ragione proteggiamo i file header sfruttando le *include guards*:

```
1  #ifndef RATIONAL_H // inizio blocco if (if not define)
2  #define RATIONAL_H // includi Rational.h
3  // ... corpo header ...
4  #endif // RATIONAL_H
```

6.2 Introduzione a CMake

Progetti complessi sono spesso composti da molti file sorgente (`.h`, `.cpp`), uno o più eseguibili e una o più librerie. Per descrivere un progetto non bastano i soli sorgenti, è necessario sapere come deve essere organizzata la compilazione.

In generale i target di compilazione sono: creare un eseguibile, creare una libreria. In entrambi i casi dobbiamo creare una lista dei file `.cpp` che devono essere compilati assieme e una lista delle librerie che devono essere linkate.

Fino ad ora abbiamo descritto i nostri progetti tramite riga di comando o tramite progetto dell'IDE. In un caso dovevamo riscrivere tutto ad ogni compilazione e dovevamo ricompilare tutti i file, nell'altro caso dipendiamo dall'IDE in uso. Cerchiamo un sistema di building standard, indipendente.

Sistema di building:

Un sistema di building è un gestore del processo di compilazione. Noi utilizzeremo CMake in quanto non è dipendente dal compilatore né dalla piattaforma utilizzata. In questo modo possiamo distribuire codice sorgente con indicazioni di building.

Introduzione all'uso di CMake: la prima cosa da fare è creare un file di testo chiamato `CMakeList.txt`, lo utilizziamo per descrivere il nostro progetto nello standard CMake. Vediamo una lista di comandi utili:

```
1  cmake_minimum_required(VERSION 2.8 FATAL_ERROR) # fisso una versione minima
    di CMake
```

```

2   project(<name> VERSION <version> LANGUAGES CXX) # dare un nome al progetto
3   add_executable(tool # target per la creazione di un eseguibile
4       main.cpp
5       another_file.cpp
6   )
7   add_library(foo STATIC # target per la creazione di una libreria
8       foo1.cpp
9       foo2.cpp
10  )

```

Un esempio:

```

1   cmake_minimum_required(VERSION 3.2 FATAL_ERROR)
2   project(tuple)
3   include_directories(include)
4   add_library(tuple
5       include/tuple.h
6       src/tuple.cpp
7   )
8   add_executable(add_tuple
9       src/add_tuple.cpp
10  )
11  target_link_libraries(add_tuple
12      tuple
13  )

```

7 Template

I template sono uno strumento che sfruttiamo per la programmazione generica. I template permettono di passare un tipo come parametro per una classe o una funzione. Il compilatore si occupa di generare il codice necessario per ogni tipo per cui il template è specializzato.

Possiamo parametrizzare una classe, in questo caso la classe prende il nome di *class template* (type generator, parametrized type o parametrized class). Il processo di generazione di una classe su un tipo specifico viene chiamato specializzazione (specialization) o *template instantiation*. Per esempio: `vector<char>` è una specializzazione di `vector`.

```

1   // file vector.h
2   template<typename T>
3   class vector {
4       int sz;
5       T* elem;
6   public:
7       vector() : sz{0}, elem{nullptr} {}
8       explicit vector(int s) : sz{s}, elem{ new T[s] } {
9           for (int i = 0; i < sz; ++i) elem[i] = 0;
10      }
11  };
12  // file vector.cpp
13  template <typename T>
14  void vector<T>::push_back(const T& d) { /* ... */}

```

Se parametrizziamo una funzione, questa prende il nome di *function template* (parametrized function o algorithm). Per esempio:

```

1  template <typename T>
2  T myMax(T x, T y) { // definisco una function template
3      return (x > y) ? x : y;
4  }
5
6  int x = i, j = 5;
7  int max = myMax<int>(i, j) // chiamo una function template
8  int max = myMax(i, j); // il compilatore deduce gli argomenti template
9  int max = myMax(22, 3.1); // errore: e' ambiguo
10 int max = myMax<int>(3.1, 2.1) // cast implicito

```

Oltre a poter passare tipi, possiamo passare literal. Per esempio potremmo sfruttare questo meccanismo con la classe `vector`: invece di utilizzare l'allocazione dinamica, utilizziamo un array con dimensione fissa (nota a tempo di compilazione). Un array con dimensione fissa può esprimere un concetto, per esempio un array di dimensione 3 esprime il concetto di tripletta: utile per rappresentare un pixel di un'immagine colorata (`vector<unsigned char, 3>`). In questi casi il valore passato deve essere un literal o una variabile costante.

Template e header:

Il compilatore (spesso) richiede che i template siano completamente definiti prima di essere usati. Siccome vogliamo che i file header contengano solo le dichiarazioni, agiamo in questo modo: mettiamo le dichiarazioni in un file con estensione `.h` (es: `vector.h`) e le definizioni in un file `.hpp` (es: `vector.hpp`); infine includiamo il file `.hpp` nell'ultima riga del file `.h`.

```

1  // file vector.h
2  #ifndef vector_h
3  #define vector_h
4  template<typename T>
5  class vector { void resize(int newsiz); };
6  #include "vector.hpp"
7  #endif // vector_h
8
9  // file vector.hpp
10 #ifndef vector_hpp
11 #define vector_hpp
12 template <typename T>
13 void vector<T>::resize(int newsiz) {}
14 #endif // vector_hpp

```

8 STL: Standard Template Library

La STL è una parte della libreria standard che fornisce contenitori e algoritmi generici. I contenitori sono molti, ognuno ha un proprio funzionamento, gli algoritmi generici ci permettono di scrivere codice che funzioni indipendentemente dal contenitore.

Un altro concetto fondamentale di STL è la sequenza: ogni sequenza ha un inizio e una fine, possiamo gestirle sfruttando degli iteratori. Gli iteratori ricordano molto puntatori a elementi di un array stile C, infatti possiamo sommarli, confrontarli, dereferenziarli e farli puntare all'elemento successivo con l'operatore `++`.

Per chiarire il concetto di algoritmo generico, vediamo un esempio:

```

1  // Restituisce il massimo elemento in [first, last)
2  template<typename Iterator>

```

```

3   Iterator high(Iterator first, Iterator last) {
4       Iterator high = first;
5       for (Iterator p = first; p != last; ++p)
6           if (*high < *p)
7               high = p;
8       return high;
9   }

```

Non è necessario conoscere i dettagli della struttura dati, basta usare i relativi iteratori. In questo modo non serve esporre tutti i dettagli della classe, evitiamo molti errori che l'utente potrebbe commettere utilizzando la nostra struttura dati.

Nota che l'algoritmo riportato ricerca l'elemento massimo che si trova tra [first, last), il last non è compreso nell'intervallo perché, per convenzione, l'iteratore last punta al primo elemento dopo l'ultimo.

Per esempio:

```

1   template<typename Elem>
2   class list { // classe che implementa iteratori
3   public:
4       class iterator; // classe in cui definiamo tutte le varie operazioni
5       iterator begin();
6       iterator end();
7       // ...
8   }
9
10  template<typename Elem>
11  class list<Elem>::iterator {
12      Link<Elem>* curr;
13  public:
14      iterator(Link<Elem>* p) : curr{p} {}
15      iterator& operator++() {...}
16      Elem& operator*() {...}
17      // ...
18  }

```

Contenitore in STL:

un contenitore:

- è una sequenza di elementi [begin() :end());
- supporta le operazioni di copia (usate nell'assegnamento o nel costruttore di copia);
- chiama il tipo dei propri elementi value_type;
- possiede tipi iteratore chiamati iterator e const_iterator
 - gli iteratori forniscono *, ++, --, ==, !=;
 - gli iteratori di vector forniscono anche [], +, -, +=, -=;
- Fornisce insert(), erase(), front(), back(), push_back(), pop_back(), size(), etc;
- forniscono operatori di confronto: ==, !=, <, <=, >, ..

9 Ereditarietà e funzioni virtuali

9.1 Ereditarietà

Definiamo *classi base* tutte le classi che raggruppano caratteristiche e comportamenti comuni a più classi. Le classi più specifiche che implementano le classi basi, ereditano le caratteristiche e i comportamenti comuni e possono implementare altre caratteristiche e comportamenti più specifici.

Per esempio una classe base potrebbe essere la classe `Shape`, essa implementa un'interfaccia comune a tutte le forme. L'effettiva implementazione dipende dalla specifica forma.

```

1  class Shape { // classe astratta
2  public: // interfaccia pubblica
3      void draw() const;
4      virtual void move(int dx, int dy);
5      Point point(int i) const; // accesso ai punti
6      int number_of_points() const;
7      Shape(const Shape&) = delete;
8      Shape& operator=(const Shape&) = delete;
9      virtual ~Shape() { } // distruttore
10 protected: // incapsulamento
11     Shape() { } // costruttore di default nascosto all'utente
12     Shape(initializer_list<Point> lst); // costruttore nascosto all'utente
13     virtual void draw_lines() const; // disegno
14     void add(Point p); // aggiunge un punto
15     void set_point(int i, Point p); // points[i] = p;
16 private: // incapsulamento
17     vector<Point> points; // non usato da tutte le Shape
18     Color lcolor; // dalla libreria grafica
19     Line_style ls; // dalla libreria grafica
20     Color fcolor; // dalla libreria grafica
21 }
```

`Shape` è una classe astratta: il costruttore di default e il costruttore con inizializzazione sono `protected`. Ciò è ragionevole, non avrebbe senso costruire oggetti di tipo `Shape`, non esiste una "forma generica". Il costruttore di copia e l'assegnamento vengono resi inutilizzabili per evitare il problema dello slicing (**comportamento obbligatorio**). Nei casi in cui si necessita di un costruttore di copia, siamo costretti ad implementare una funzione `clone()`.

Il problema dello slicing:

Lo slicing si verifica quando tentiamo di inserire una classe derivata in uno slot che riesce a contenere solo membri della classe base. Il compilatore risolve il problema affettando l'oggetto, lasciando solo i membri della classe base.

Per esempio: poniamo di avere creato la classe che rappresenta una circonferenza `class Circle : public Shape {...}`; che estende `Shape`. Se non eliminassimo il costruttore di copia, potremmo eseguire il seguente frammento di codice:

```

1  void my_fct(const Circle& c) {
2      vector<Shape> v;
3      v.push_back(c);
4  }
```

I dati membro aggiuntivi di `c` (per esempio il raggio) definiti in `Circle` vengono persi, rimangono solo i membri che sono stati dichiarati in `Shape`.

9.2 Funzioni virtuali

Le funzioni virtuali danno la possibilità di definire la stessa funzione nella classe base e in quella derivata. Questo processo viene chiamato polimorfismo runtime in quanto, la funzione che deve essere eseguita, viene determinata a tempo di esecuzione.

Una funzione dichiarata `virtual` in una classe base può essere ridefinita in una classe derivata. All'interno della classe derivata, la funzione da sovrascrivere, deve avere stesso nome, stessi tipi, e stessa constness della funzione della classe base. Inoltre è possibile (non obbligatorio) dichiarare esplicitamente l'override di una funzione, basta aggiungere `override` alla dichiarazione della funzione (nella classe derivata) (il compilatore controlla se è lecito).

```

1  class D : public B {
2      void f() const override;
3      void g() const override;
4  };

```

Se vogliamo imporre che tutte le classi derivate implementino la funzione virtuale, allora questa deve essere posta uguale a zero nell'header della classe base. Per esempio:

```

1  class B {
2  public:
3      virtual void f() = 0; // richiesto overriding
4      virtual void g() = 0; // richiesto overriding
5  };

```

In questo caso la funzione viene chiamata virtuale pura. Le classi astratte composte da solo funzioni virtuali pure (comportamento molto comune) sono delle interfacce pure, cioè non presentano né data member né costruttori. Non c'è la necessità di mascherare il costruttore all'utente, infatti non è possibile istanziare oggetti.

9.3 Virtual pointer e virtual table

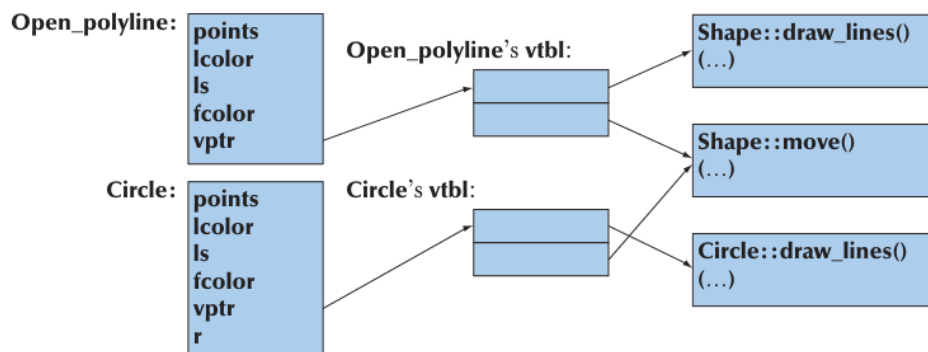


Figura 1: virtual table e virtual pointer

Facendo sempre riferimento alla classe `Shape`. Abbiamo due classi derivate che implementano `Shape`. Le chiamate a funzioni virtuali vengono gestite dalle virtual table (vtbl) e dai virtual pointer (vptr): ogni classe ha una propria tabella contenente tutte le possibili funzioni che possiamo chiamare e un puntatore che indica quali funzioni devono essere chiamate.

In questo modo il compilatore riesce a individuare l'esatta funzione da chiamare: se una classe derivata (es: `Circle`) ha sovrascritto una funzione (`draw_lines()`), allora il puntatore indica che deve essere chiamata la funzione sovrascritta (`Circle::draw_lines()`); se la

funzione non è stata sovrascritta (`move()`), il puntatore indica che deve essere chiamata la funzione della classe madre (`Shape::move()`).

Un altro esempio: abbiamo creato un oggetto di tipo `Marked_polyline` su cui facciamo una chiamata alla funzione `move()`. Se `move()` non è stata sovrascritta, allora il `vp` di `Marked_polyline` punterà a `Open_polyline::move()`; se `Open_polyline` ha sovrascritto la funzione verrà chiamata `Open_polyline::move()`, altrimenti il suo `vp` punterà a `Shape::move()` (che a questo punto verrà chiamata).

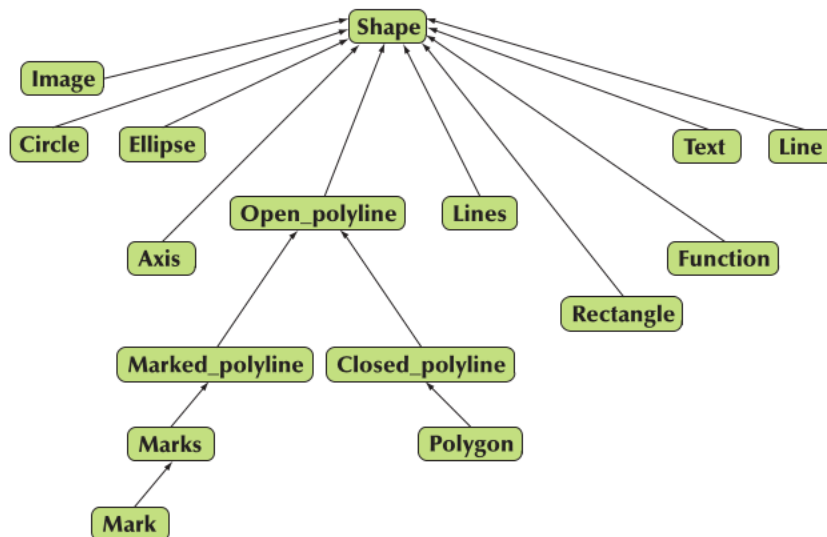


Figura 2: Ereditarietà di classe

Negli esempi riportati abbiamo considerato solo funzioni virtuali. Le funzioni "normali" (quelle non virtuali) non vengono gestite runtime, vengono gestite in fase di compilazione. In ogni caso anche le chiamate a queste funzioni sfruttano puntatori che puntano al codice che deve essere eseguito.

9.4 Navigare nella gerarchia di classi

L'*upcasting* (da classe derivata a base) è sempre lecito, "Circle is a Shape", non necessita di cast esplicito. I *downcast* (da classe base a derivata) devono essere esplicitati con `dynamic_cast`, è l'unico tipo di cast che viene gestito run-time. Funziona con i puntatori (se non lecito torna un `nullptr`) e con le reference (se non lecito lancia `bad_cast`).

9.5 Polimorfismo

Definiamo polimorfismo la capacità di associare comportamenti diversi alla stessa notazione generica. Abbiamo visto due strumenti che causano un comportamento di questo tipo: l'ereditarietà accoppiata alle funzioni virtuali e i template. Questi due strumenti sono ben diversi tra loro, causano due differenti "tipi" di polimorfismo. Si parla di polimorfismo dinamico, polimorfismo statico.

Polimorfismo dinamico: viene gestito tramite ereditarietà e funzioni virtuali. Permettono di identificare un insieme comune di funzionalità tra classi collegate. Nella classe base viene definita l'interfaccia comune, ciascuna classe derivata sovrascrive le operazioni in base alle proprie specifiche. Con questo meccanismo possiamo riutilizzare lo stesso codice per gestire istanze di classi diverse che implementano comportamenti simili.

Il polimorfismo dinamico è

- vincolato: le classi derivate sono vincolate all'interfaccia della classe base;
- dinamico: viene gestito run-time in base al tipo dell'oggetto.

Polimorfismo statico: viene gestito tramite template. Presuppone che la stessa sintassi sia condivisa dalle varie istanze del template, classi e funzioni vengono definite in maniera indipendente ma devono condividere la stessa sintassi. Il polimorfismo si concretizza specificando la classe o la funzione con vari tipi diversi.

Il polimorfismo statico è

- svincolato: l'interfaccia delle classi derivate non è predeterminata;
- statico: viene gestito a tempo di compilazione.

10 Introduzione agli algoritmi STL

Andiamo ad analizzare alcuni algoritmi della STL. La STL ne offre circa 80, vediamo alcuni algoritmi principali per familiarizzare con la sintassi, questo offre una buona base di partenza per poterne utilizzare un buon numero.

10.1 Un primo esempio: find()

Permette di ricercare un elemento in una sequenza.

```

1  template<typename In, typename T>
2  // In: iteratore di input
3  // T deve essere confrontabile
4  In find(In first, In last, const T& val) {
5      while(first != last && *first != val) ++first;
6      return first;
7  }
```

Torna un iteratore che punta all'elemento trovato, se l'elemento non è presente viene tornato un puntatore a last.

L'algoritmo è indipendente dal tipo di container e dal tipo di elemento da ricercare.

10.2 Algoritmi con predicati: find_if

L'algoritmo find_if() verifica che un criterio sia soddisfatto.

```

1  template<typename In, typename Pred>
2  // In: iteratore di input
3  // Pred e' un predicato
4  In find_if(In first, In last, Pred pred) {
5      while(first != last && !pred(*first)) ++first;
6      return first;
7  }
```

Il criterio viene espresso da un predicato, ovvero una funzione che torna `true` o `false`. Per esempio:

```

1  bool odd(int x) { return x%2; } // definisco il predicato
2
3  void f(vector<int>& v) {
```

```

4      auto p = find_if(v.begin(), v.end(), odd); // odd e' un puntatore che
        punta alla funzione odd()
5      if (p != v.end()) { /* numero dispari */ }
6  }

```

Il puntatore a una funzione è una peculiarità del linguaggio C, è possibile ricavare il puntatore di qualsiasi funzione.

Tuttavia questo metodo non permette di esprimere qualsiasi criterio, possiamo usarlo se non dobbiamo passare parametri alla funzione definita. Prendiamo in considerazione il criterio "maggiore di un dato valore": in questo caso ci serve un modo per passare alla funzione il valore da confrontare con gli elementi di un contenitore.

Per questo tipo di casi necessitiamo di un **function object**: un oggetto che può essere usato come una funzione. Possiamo definire un predicato nel seguente modo:

```

1      class Larger_than {
2          int v;
3      public:
4          Larger_than(int vv) : v{vv} {} // salvataggio argomento
5          bool operator() (int x) const { return x>v; } // confronto
6      };
7
8      void f(list<double>& v, int x) {
9          auto q = find_if(v.begin(), v.end(), Larger_than(x)); // creo il function
        object
10         if (q != v.end()) { /* valore trovato */ }
11     }

```

L'`operator()` viene chiamato function call operator o application operator. Nell'algoritmo `find_if()` la chiamata `!pred(*first)` verrà tradotta in una chiamata a `operator()`.

Passiamo i function object per copia, se sono più grandi di qualche byte è più efficiente passarli per riferimento.

Questo tipo di combinazione è più efficiente di una chiamata a funzione.

10.3 `sort()`, `copy()` e `lambda expression`

I function object hanno due debolezze: sono definiti in una regione di codice; vengono usati altrove. Usando le `lambda expression` aggiriamo questi vincoli. Le **lambda expression** (o `lambda function`) permettono di definire function object sul posto, permettono di crearne uno immediatamente e usarlo come argomento di una funzione. Questo tipo di function object non può essere utilizzato altrove.

`sort()`:

effettua un ordinamento tra due iteratori. Esistono due versioni:

```

1      // ordinamento con operator<
2      template<typename Ran>
3          // Ran deve possedere un random_access_iterator
4      void sort(Ran first, Ran last);
5
6      // ordinamento con funzione dedicata
7      template<typename Ran, typename Cmp>
8          // Ran deve possedere un random_access_iterator
9          // Cmp deve essere una funzione di ordinamento su Ran
10     void sort(Ran first, Ran last, Cmp cmp);

```

Se vogliamo ordinare un intero contenitore dobbiamo usare l'algoritmo `sort()` che non è parte della STL, si trova in `std_lib_facilities.h`.

```

1  template<typename C>
2  // C e' un contenitore
3  void sort(C& c) {
4      sort(c.begin(), c.end());
5  }
```

Vediamo un esempio con l'algoritmo `sort()` e una lambda expression.

```

1  // vogliamo ordinare un vettore di Record: vector<Record> vr;
2  struct Record {
3      string name;
4      char addr[24];
5  };
6  // possiamo ordinarlo in base a due diversi predicati
7  struct Cmp_by_name {
8      bool operator()(const Record& a, const Record&b) const
9      { return a.name < b.name; }
10 };
11 struct Cmp_by_addr{
12     bool operator()(const Record& a, const Record&b) const
13     { return strcmp(a.addr, b.addr, 24) < 0; }
14 };
15 // possiamo gestire la situazione con le lambda expression:
16 sort(vr.begin(), vr.end(),
17     [] (const Record& a, const Record& b)
18     { return a.name < b.name; }
19 );
20 sort(vr.begin(), vr.end(),
21     [] (const Record& a, const Record& b)
22     { return strcmp(a.addr, b.addr, 24) < 0; }
23 );
```

Le lambda expression accettano argomenti come le funzioni, inoltre possono catturare le variabili locali. Per esempio:

```

1  void func5(std::vector<double>& v, const double& epsilon) {
2      std::transform(v.begin(), v.end(), v.begin(),
3          [epsilon](double d) -> double { // cattura la variabile epsilon e
4              // specifico che torno un tipo double (opzionale)
5              if (d < epsilon) return 0;
6              else return d;
7          });
8  }
```

Capture clause	Effetto
[&epsilon, zeta]	Cattura epsilon per riferimento e zeta per valore (copia)
[&]	Cattura tutte le variabili usate nella lambda expression per riferimento
[=]	Cattura tutte le variabili usate nella lambda expression per valore
[&, epsilon]	Cattura tutte le variabili usate nella lambda expression per riferimento, ma epsilon è per valore
[=, &epsilon]	Cattura tutte le variabili usate nella lambda expression per valore, ma epsilon è per riferimento

Figura 3: La cattura delle variabili locali

copy():

effettua una copia.

```

1  template<typename In, typename Out>
2  // In: iteratore di input
3  // Out: iteratore di output
4  Out copy(In first, In last, Out res) {
5      while(first != last) {
6          *res = *first;
7          ++res;
8          ++first;
9      }
10     return res;
11 }
```

Viene effettuata una copia di una sequenza in un'altra sequenza, i container possono essere diversi. Non viene effettuato nessun controllo sullo spazio necessario, spetta all'utente verificare. Esiste anche `copy_if()`, necessita di un predicato come quarto argomento.

10.4 Associative container**Map:**

esempio di utilizzo:

```

1  // conteggio parole ripetute
2  int main() {
3      map<string, int> words;
4      for(string s; cin >> s; )
5          ++words[s]; // se la parola non esiste viene creato un elemento con
                      // chiave 1
6      for(const auto& p : words)
7          cout << p.first << ": " << p.second << '\n';
8      return 0;
9  }
```

Gli elementi di una mappa sono coppie chiave valore, in particolare sono oggetti `pair<T1, T2>`. Creiamo oggetti coppie con `make_pair()` e accediamo ai membri con `p.first` e `p.second`.

```

1  cout << p.first << ": " << p.second << '\n';
```

Set:

un set è una mappa senza valori, contiene solo chiavi. Viene implementato con una struttura ordinata per poter ottenere prestazioni ottime. Quindi dobbiamo definire una funzione d'ordine se vogliamo memorizzare UDT in un set.

11 Smart pointer

11.1 Risorse ed eccezioni

Le eccezioni possono corrompere la gestione delle risorse. Con risorse intendiamo: memoria; lock; file aperti; thread; socket; window; etc.

Analizziamo il seguente codice:

```

1 void suspicious(int s, int x) {
2     int* p = new int[s];
3     vector<int> v;
4     // ...
5     if(x) p[x] = v.at(x);
6     // ...
7     delete[] p;
8 }
```

Non siamo sicuri che la memoria venga rilasciata. La funzione `at()` potrebbe lanciare un'eccezione e l'istruzione `delete[] p` potrebbe quindi non essere mai eseguita.

Per risolvere il problema potremmo gestire l'eccezione in un blocco `catch`, liberare le risorse e rilanciare l'eccezione con `throw`. Questo tipo di soluzione rende il codice verboso e ripetitivo.

Questo problema è molto comune se scegliamo di gestire la memoria dinamica "manualmente", se utilizzassimo (per esempio) gli `std::vector` non dovermo preoccuparci di questi potenziali leak. Ma come fa `std::vector` a gestire questo tipo di situazioni? La risposta è semplice: la memoria acquisita in fase di costruzione viene liberata in fase di distruzione (dal distruttore). Quando un flusso di esecuzione (chiamato thread) lascia uno scope, vengono invocati di conseguenza i distruttori di tutti gli oggetti e sotto-oggetti.

Questa tecnica prende il nome di **Resource Acquisition Is Initialization (RAII)**, è un'idea generale sfruttata anche nei socket, I/O buffer (inclusi i file), etc.

Analizziamo un altro caso:

```

1 // esempio didattico: non si alloca dinamicamente un vector
2 vector<int>* make_vec() {
3     vector<int>* p = new vector<int>;
4     // ...riempimento del vettore con i dati...
5     // Potrebbe ritornare un'eccezione
6     return p;
7 }
```

Possono insorgere problemi se viene lanciata un'eccezione durante il riempimento. La funzione `make_vec()` deve restituire un puntatore se l'inserimento dei dati non genera eccezioni, in caso contrario la funzione lancerà un'eccezione (non esegue il `return`) ma non dovrà lasciare memory leak. Dobbiamo utilizzare un blocco `try/catch` come abbiamo descritto sopra:

```

1 vector<int>* make_vec() {
2     vector<int>* p = new vector<int>;
3     try {
```

```
4      // ... riempimento del vettore con i dati
5  } catch(...) { // catturo qualsiasi eccezione
6      delete p;
7      throw; // rilancio l'eccezione catturata
8  }
9  return p;
10 }
```

Questo tipo di tecnica prende il nome di *basic guarantee*, si tratta di una garanzia fornita dalla funzione. Esistono tre tipi di garanzie che le funzioni possono fornire:

- **basic guarantee:** la funzione gestisce l'eccezione/i oppure lancia l'eccezione e libera la memoria. STL fornisce questo tipo di garanzia;
- **strong guarantee:** la funzione rispetta la basic guarantee, in più tutti i valori osservabili (valori non locali alle funzioni) sono gli stessi che erano presenti prima della chiamata, anche in caso di fallimento;
- **no-throw-guarantee:** la funzione non lancia eccezioni.

L'esempio che abbiamo visto utilizzava un blocco `try/catch` per offrire una basic guarantee. Esiste un modo più elegante per farlo: *smart pointer*.

11.2 Smart pointer

Sono classi template definite in `<memory>`, gestiscono automaticamente la deallocazione della memoria quando escono dallo scope. In questo modo si evitano memory leak e dangling pointer. Esistono tre tipi di smart pointer:

- `unique_ptr`: dealloca la memoria uscendo dallo scope; permette i move; non permette copie (lo spostamento è consentito). L'utilizzo è simile a quello di un puntatore normale, ma implementa due funzioni interessanti: con `reset()` possiamo re-inizializzarlo, se deteneva un puntatore prima della chiamata, l'oggetto puntato viene distrutto; con `release()` estrare il puntatore dallo `unique_ptr`, il quale viene resettato a `nullptr` (permette di estrarre il puntatore incapsulato). Esempio:

```
1 // esempio didattico: non si alloca dinamicamente un vector
2 vector<int>* make_vec() {
3     unique_ptr<vector<int>> p { new vector<int> };
4     // ... riempimento del vettore con i dati ...
5     // Potrebbe lanciare eccezioni
6     return p.release();
7 }
```

In questo caso evitiamo memory leak infatti, se viene lanciata l'eccezione, viene chiamato il distruttore di `unique_ptr` che dealloca la memoria. Vediamo un caso particolare:

```
1 void no_good() {
2     int* p = new int;
3     unique_ptr<int> sp {p};
4     unique_ptr<int> sp2 {p}; // errore logico ma compila
5 } // quando esce dallo scope, entrambi i puntatori eliminano l'oggetto
```

Quindi, inizializzare due `unique_ptr` con lo stesso puntatore è un errore (logico non rilevato). Per ovviare a questo problema, si può utilizzare la funzione `make_unique`.

- `shared_ptr`: funziona in un modo simile a `unique_ptr` ma rimuove alcuni vincoli. Uno `shared_ptr` può essere copiato, può essere condiviso, cioè molti `shared_ptr` possono detenere lo stesso puntatore.

Gli `shared_ptr` sfruttano un algoritmo di *reference counting*, in questo modo gli `shared_ptr` che detengono lo stesso puntatore si conoscono. La memoria viene deallocata solo quando l'ultimo `shared_ptr` che detiene la memoria viene distrutto.

Questo meccanismo è molto utile ma consuma risorse. Esempio di utilizzo:

```
1  int main() {  
2      shared_ptr<int> moved_ptr = AllocateAndReturn();  
3      shared_ptr<int> copied_ptr = moved_ptr; // copia permessa  
4      return 0;  
5  }
```

- `auto_ptr` (deprecated);